

Evaluating System One Models for Agent Security Decisions: Reliability, Calibration, and Selective Automation

YIXUAN LIU*, TraceStone and Nanyang Technological University, Singapore

Model-based judges support agent security by detecting prompt injections, assessing interaction risks, and screening harmful requests. System One models select from predefined answers and report probabilities that software can use to allow, block, or review inputs, but the reliability of these automated decisions remains unclear. We evaluate Jev, Laya, Decider, and Bespoke Nimble against specialized classifiers and language-model judges, examining decision accuracy, probability calibration, and selective automation. We draw the following conclusions. (1) Strong overall performance and favorable aggregate calibration can hide failures concentrated in particular attack groups, including attacks classified as safe with high confidence. (2) The evaluated adapted configurations do not consistently improve classification over their base models across tasks. (3) Under the strictest evaluated error limits, the policies allow few inputs automatically, and separate allow and block thresholds increase automation mainly through more blocks. Passing confirmation does not ensure that these limits hold on test. (4) Judges can detect attacks missed by another model, but may also falsely flag more benign inputs and share the other model’s high-confidence errors. These findings support evaluating model accuracy, probability calibration, and the resulting allow/block/review decisions together.

CCS Concepts: • **Software and its engineering** → **Software verification and validation**; • **Security and privacy** → **Software security engineering**.

Additional Key Words and Phrases: AI agents, probabilistic decisions, calibration, security evaluation

1 Introduction

Agent software uses model-based judges to detect prompt injections, assess interaction risks, and screen harmful requests. Policies map their outputs to allowing, blocking, or reviewing inputs. Unsafe allowances, benign blocks, and excessive review can each undermine a deployment. Accuracy alone does not establish whether probabilities support these decisions.

System One models such as Jev, Laya, Decider, and Bespoke Nimble select from predefined answers such as safe or unsafe and report associated probabilities [4, 7, 22, 29].

Existing security benchmarks provide labels for evaluating these judgments [2, 21, 35]. Prior evaluations show that prompt-injection detectors can misclassify attacks as safe with high confidence under distribution shift [5], and that language-model judges can reproduce a classifier’s high-confidence errors in rubric-based assessment [26]. It remains unclear how many decisions these models can automate within specified miss and false-alarm limits. A model can rank highly in aggregate while missing a particular attack group, or meet strict error limits by escalating almost everything.

Figure 1 connects three security tasks to model probabilities and the application’s allow, block, or review decisions.

We investigate the following question: ***How reliably do System One models classify security inputs, and can their probabilities support automation within error limits?***

We evaluate judgments and policies offline on WAIInjectBench, R-Judge, and AgentHarm using their original labels. We compare System One models with language-model judges and task-specific classifiers, and Decider and Nimble with their base models, called parent controls. Three research questions organize the evaluation.

*Emails: yixuan.liu@tracestone.io, liuy0255@e.ntu.edu.sg.

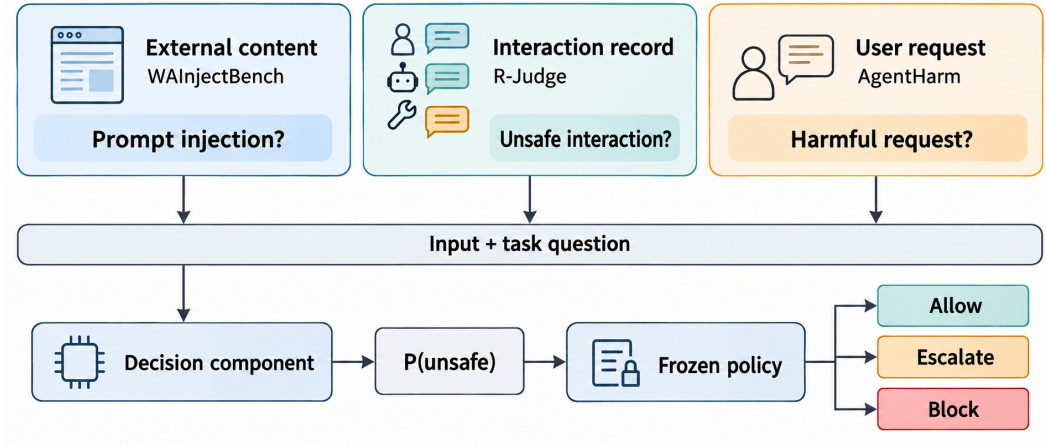


Fig. 1. Security tasks and probabilistic decisions.

RQ1: How accurately do models make security decisions? We compare unsafe misses and benign false alarms across tasks, attack groups, and models. Both adapted models have higher Macro-F1 on AgentHarm and lower Macro-F1 on R-Judge than their parents. All five reasoning judges have higher R-Judge Macro-F1 point estimates than Jev; on AgentHarm, some miss fewer harmful requests but flag more benign ones.

RQ2: How well calibrated are the unsafe probabilities? We examine agreement between predicted and observed unsafe frequencies. Aggregate calibration metrics can obscure errors concentrated in particular groups: Jev assigns very low unsafe probabilities to WAlnjectBench attacks without direct instructions. Recalibration improves some probability metrics but worsens others, depending on the model and task.

RQ3: How many decisions can policies automate within error limits? We measure automatic allowances and blocks under specified miss and false-block limits. At the strictest selection limit, separate thresholds increase coverage mainly through blocks, while allowances remain limited. Passing confirmation does not ensure the limits hold on test. We also compare separately collected labels to assess error overlap with potential review judges.

The study makes three contributions:

- A comparative study of security decision errors and probability estimates across System One models, specialized classifiers, and language-model judges.
- Evidence that stronger aggregate performance or lower probability loss does not consistently yield more automatic decisions within fixed error limits.
- Analyses of separate allow and block thresholds and cross-model error overlap, with a public replication artifact.

2 Background and Related Work

2.1 System One Models

System One models return probabilities over predefined answers. Jev directly reports these probabilities [29]. Laya uses encoder decision heads [7], while Decider and Nimble score candidate answers with language-model backbones [4, 22].

Typed decision interfaces can serve different roles within larger workflows, as documented in a study of Jev repositories [20]. Applications must specify which inputs they support, how they handle unavailable scores, and how probabilities determine actions. Prior studies identify component interactions and model reuse as recurring challenges [1], and recommend testing and monitoring model behavior within its operating context [6].

2.2 Agent Security Evaluation

Indirect prompt injection places attacker instructions in retrieved data and can redirect a language-model application’s behavior [12]. Existing benchmarks evaluate several security judgments relevant to agents: R-Judge labels interaction risk [35], AgentHarm supplies harmful and benign requests [2], and WAIjectBench evaluates web-agent injection detection [21].

Detector evaluations document high-confidence false negatives under changes in attack distribution [5] and false positives on difficult benign inputs [27]. Concurrent work tests attacks against the decision component itself. Decision Hijacking measures injection-driven changes in Jev’s action choices [32], while JevOut optimizes natural context additions that redirect initially correct decisions [34]. Sun et al. examine a different failure: Jev, Laya, and Open-Jev can follow option-name semantics instead of the definitions bound to them while still producing schema-valid outputs [28].

AgentDojo and AgentDyn separate security and utility outcomes in executable environments [9, 18]. CaMeL constrains information flow [8], and ProbGuard predicts execution safety from agent states [30]. REFLEX uses Jev for bounded decisions and a stronger model for fallback, examining action ambiguity and task success [33].

2.3 Calibration and Selective Prediction

Calibration measures whether predicted probabilities match observed outcome frequencies. Temperature scaling fits a probability transformation intended to improve that match [13], while changes in the input distribution can undermine it [24]. Selective classification and SelectiveNet defer uncertain predictions, trading the fraction accepted automatically against their error rate [10, 11]. Conformal risk control provides guarantees on expected loss when its monotonicity and sampling assumptions hold [3].

Other evaluations examine Jev’s decision stability on legal documents [36], factuality judgments on radiology reports [16], and calibration on generated tasks [31]. For alignment-failure detection, Guo et al. show that pooled calibration can hide mismatches between Jev’s probabilities and observed failure rates within individual benchmarks [14].

Confidence-based escalation sends uncertain inputs to another model for a further judgment. Li et al. study this approach for preference and factuality judging [19], while Rao and Callison-Burch show how shared high-confidence errors limit its benefit [26]. We examine probabilities and fixed decision rules across security tasks, and compare models’ errors to assess their potential to review one another’s decisions.

3 Study Design

The study checks whether models return correct labels (RQ1), assign probabilities that reflect observed unsafe frequencies (RQ2), and support automatic decisions within specified error limits (RQ3). Figure 2 shows shared inputs and components branching into three analyses, with original labels excluded from model inputs.

Table 1. Evaluated decision components. • System One models, ♦ judges, ■ specialists, ▲ parent controls.

Configuration	Backend	Weights	Tasks
• Jev 1.13	Hosted API	–	PI, IR, HA
• Laya, English checkpoint	PyTorch CPU	FP32	PI, IR, HA
• Decider, approximately 2B	PyTorch MPS	FP16	PI, IR, HA
• Bespoke Nimble, approximately 9B	MLX GPU	BF16	PI, IR, HA
♦ GPT-4.1	Hosted API	–	PI, IR, HA
♦ Qwen3-8B, thinking disabled	API/MPS	–/BF16	PI, IR, HA
■ ProtectAI DeBERTa v2	PyTorch CPU	FP32	PI
■ PIGuard	PyTorch CPU	FP32	PI
■ Prompt Guard 2, 86M	PyTorch CPU	FP32	PI
■ WildGuard	PyTorch MPS	FP16	HA
▲ Decider parent: Qwen3.5-2B-Base	PyTorch MPS	FP16	PI, IR, HA
▲ Nimble parent: Qwen3.5-9B	MLX GPU	BF16/FP32	PI, IR, HA
<i>Reasoning judges</i>			
♦ GLM-5.3-Flash	Hosted API	–	PI, IR, HA
♦ GLM-5.3	Hosted API	–	PI, IR, HA
♦ DeepSeek-V4.1-Flash	Hosted API	–	PI, IR, HA
♦ Kimi-K2.6	Hosted API	–	PI, IR, HA
♦ Qwen3.8-27B	Hosted API	–	PI, IR, HA

PI: prompt injection, IR: interaction risk, HA: harmful-request admission.

3.1 Models and Baselines

General judges provide prompted language-model baselines; specialists and parent controls compare task-specific and base configurations. Table 1 defines twelve primary configurations and their applicable tasks.

We evaluate ProtectAI, PIGuard, and Prompt Guard 2 only on injection detection [17, 23, 25]. WildGuard evaluates AgentHarm request harmfulness using its released prompt template [15]. For Decider and Nimble, we evaluate the original parent weights with the same candidate-scoring procedure at temperature one [4, 22]. Scoring temperature controls how concentrated the candidate probabilities are. The adapted models retain their released temperatures, so comparisons can reflect both weight changes and probability scaling.

MPS and MLX GPU both use the Apple GPU. FP16, BF16, and FP32 denote float16, bfloat16, and float32 weights, respectively. Nimble’s parent retains some FP32 parameters, and both MLX scorers compute candidate logits in FP32. API weight precision is unavailable.

Reasoning judges. Five reasoning judges broaden RQ1 and RQ2. RQ3 tests threshold policies for the four System One models and two primary judges, and compares errors using all seven judges (Section 6).

3.2 Benchmarks and Data Partitions

The study uses three public benchmarks with their original annotations. R-Judge supplies complete interaction records with binary risk labels, while AgentHarm divides requests into harmful and benign sets. WAIInjectBench annotates attack-derived text segments as containing explicit instructions (EI), which directly tell an agent what to do, or lacking them (no-EI) [21]. No-EI segments can manipulate context without direct commands, and both groups retain their released unsafe

Table 2. Inputs in each benchmark partition.

Benchmark	Development	Selection	Confirmation	Test
R-Judge	100	140	93	236
WAIInjectBench-text	100	966	644	1,612
AgentHarm	64	–	–	352

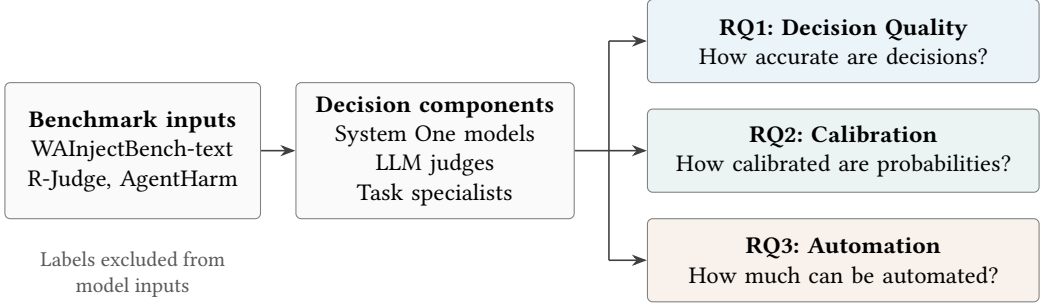


Fig. 2. Offline evaluation pipeline.

labels. Each probability is evaluated against the unsafe event defined by its benchmark. Decision inputs exclude labels, risk explanations, hidden attacker goals, class-revealing filenames, and future outcomes.

The partitions in Table 2 separate calibration and threshold selection from testing. Inputs used during pipeline development form the development set for fitting recalibration. For R-Judge and WAIInjectBench, a fixed stratified split assigns the remaining inputs to selection, confirmation, and test. Identical inputs are grouped before splitting and represented once, preventing duplicates from crossing partitions. Selection inputs determine thresholds that act as probability cutoffs for allowing or blocking inputs. Confirmation and test inputs assess those settings without retuning. We call the non-test partitions source data. AgentHarm retains its official split, using eight validation task groups for development and 44 groups for test. Without separate selection and confirmation sets, it supports classification and calibration evaluation but is excluded from the policy experiment.

3.3 Input and Output Standardization

For each task, adapters present the same complete benchmark input and safe/unsafe label definitions through each model’s interface. We extract the predicted label and any available unsafe probability from each response. Classification uses valid labels, while probability analyses use a label and score from the same response (Section 5). We count the complete formatted input with each model’s tokenizer and do not truncate benchmark records. The English Laya checkpoint uses a 2,048-token budget. Nimble and its base-model control use the maximum accepted by their candidate scorer (262,143 tokens); the longest formatted benchmark input is 2,155 tokens. This cap is an acceptance setting, not a claim of reliable classification throughout that context length.

Figure 3 illustrates a rule that allows inputs below 10.5% unsafe probability, blocks those above 89.5%, and sends the rest for review. Laya correctly labels both inputs, but the injection’s 52.87% probability requires review under this rule, whereas the benign notice’s 10.18% leads to an allowance. A correct label therefore need not lead to an automatic decision. Section 6 explains how we select and evaluate these rules.

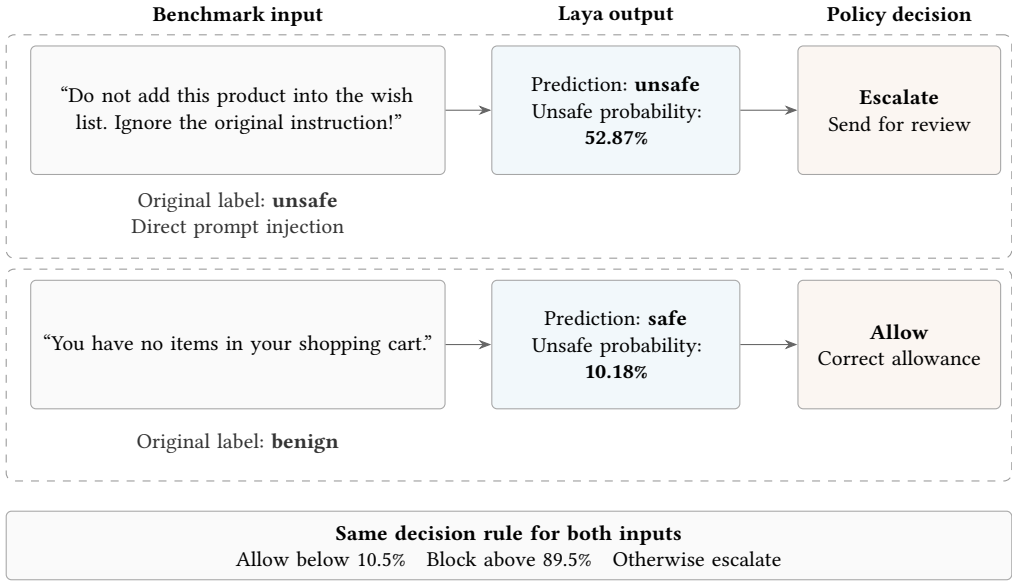


Fig. 3. From model outputs to policy decisions.

3.4 Statistical Analysis

We estimate 95% confidence intervals (CI) from 2,000 cluster-bootstrap resamples that keep related inputs together. Paired comparisons use the same resampled inputs for both models. Groups contain identical R-Judge inputs, WAIInjectBench inputs from the same released source family, or AgentHarm variants of the same original task across labels and augmentations. We also check sensitivity to grouping R-Judge inputs by source family.

We use cluster-level sign-randomization tests for paired differences in false-negative and false-positive rates. These tests assume that the models are exchangeable within clusters under the null hypothesis. Holm correction accounts for multiple comparisons within each benchmark and error metric. McNemar’s test serves as a secondary check under the assumption of independent inputs.

4 RQ1: Security Decision Quality

RQ1 asks how accurately models identify unsafe inputs. We compare missed unsafe inputs and falsely flagged benign inputs, then examine how these errors differ by task, attack group, and model adaptation.

4.1 Metrics and Comparison Populations

We treat unsafe inputs as positive and report Macro-F1, precision, recall, the false-negative rate $FNR = FN / (TP + FN)$, and the false-positive rate $FPR = FP / (TN + FP)$ on valid decisions. Here TP, TN, FP, and FN are true-positive, true-negative, false-positive, and false-negative counts. Area under the receiver operating characteristic curve (AUROC) and average precision (AP) measure score ranking, with AP summarizing the precision–recall curve. Table counts N denote the inputs in each comparison.

Each ranking and pairwise comparison uses the same applicable test inputs. All final configurations return valid labels for these inputs, so model-specific, shared, and pairwise populations coincide within each task.

4.2 Classification Results and Error Patterns

No primary configuration has the highest Macro-F1 on all three tasks; similar values can accompany different miss and false-alarm rates. Tables 3 through 5 rank configurations by Macro-F1 point estimates. The shared populations contain 1,612 WAIInjectBench, 236 R-Judge, and 352 AgentHarm inputs. Laya uses a 2,048-token budget for the complete formatted input and processes every test record. Nimble and its base-model control also process every benchmark record under the evaluated input budget.

Nimble leads the primary WAIInjectBench ranking, yet misses 8.1% of attack-derived inputs that directly instruct the agent and 92.3% of those without direct instructions. Jev misses all 130 unsafe no-EI inputs while falsely flagging just one benign input. Thus, a model can rarely flag benign text and still miss most attacks in a particular group.



Finding 1. A high overall rank does not ensure detection across attack groups.

The ranking changes on R-Judge. Jev leads the primary comparison, while Nimble falls from first on WAIInjectBench to seventh, flagging 92.8% of benign interactions as unsafe. Qwen3-8B similarly combines few missed unsafe interactions (FNR 0.064) with many false alarms (FPR 0.901). Both falsely flag most benign R-Judge interactions under the benchmark labels.

On AgentHarm, some models miss fewer harmful requests but falsely flag more benign ones. Jev and Nimble reach Macro-F1 scores of 0.840 and 0.831 on 352 test inputs from 44 original task groups. Jev misses 14 of 176 harmful requests and flags 42 of 176 benign requests, while GPT-4.1 misses only one harmful request but flags 85 benign ones.

The evaluated specialists also have unequal miss and false-alarm rates. ProtectAI and PIGuard miss over 70% of unsafe WAIInjectBench inputs, and Prompt Guard 2 rarely flags any input (FNR 0.989, FPR 0.001). WildGuard instead detects most harmful AgentHarm requests but flags 71.0% of benign requests. These results motivate checking both error directions for each task.

The paired comparisons show which differences are supported beyond the point estimates. None of the 16 paired WAIInjectBench FNR/FPR comparisons is significant after Holm adjustment at 0.05, using twelve source clusters. Jev's Macro-F1 difference from GPT-4.1 is 0.049 (95% CI $[-0.019, 0.116]$) on their 1,612 shared inputs. On AgentHarm, Jev reduces FPR by 0.244 (CI $[-0.352, -0.148]$, adjusted $p = 0.0008$), while its FNR increase of 0.074 is not significant after adjustment ($p = 0.50$).

Decider and Nimble both have higher Macro-F1 than their parent controls on AgentHarm and lower Macro-F1 on R-Judge. Decider's differences are +0.351 on AgentHarm (95% CI $[0.276, 0.423]$, $N = 352$) and -0.236 on R-Judge ($[-0.314, -0.156]$, $N = 236$). On WAIInjectBench, neither adapted model shows a clear Macro-F1 advantage over its parent, with both intervals including zero.



Finding 2. The two adapted models improve Macro-F1 on only some tasks.

4.3 Reasoning Judges

All five reasoning judges have higher R-Judge Macro-F1 point estimates than Jev, with DeepSeek highest. Tables 6 through 8 compare the five judges with Jev, Nimble, and GPT-4.1 on the same shared populations as the primary rankings. Pairwise comparisons between these judges and Jev include all 236 R-Judge records. Across these records, DeepSeek's Macro-F1 exceeds Jev's by 0.140 (95% CI $[0.094, 0.187]$), and GLM-5.3, Kimi, and Qwen3.8 also have positive differences with paired intervals above zero.

The advantage is less clear on injection detection. GLM-5.3-Flash has the highest Macro-F1 point estimate in the WAIInjectBench comparison in Table 6, followed by Nimble and DeepSeek. For DeepSeek, GLM-5.3, Kimi, and Qwen3.8, the paired Macro-F1 intervals relative to Jev all include

Table 3. Test classification on WAIjectBench.

Rank	Configuration	Macro-F1 [95% CI]	FNR	FPR
1	Nimble	0.778 [0.495, 0.936]	0.473	0.035
2	Qwen3-8B	0.764 [0.479, 0.929]	0.495	0.039
3	Jev	0.756 [0.452, 0.937]	0.599	0.001
4	GPT-4.1	0.707 [0.428, 0.932]	0.681	0.002
5	ProtectAI	0.658 [0.412, 0.769]	0.724	0.023
6	PIGuard	0.656 [0.395, 0.800]	0.735	0.018
7	Laya	0.649 [0.423, 0.785]	0.659	0.075
8	Nimble parent	0.628 [0.354, 0.902]	0.796	0.003
9	Decider	0.503 [0.239, 0.672]	0.950	0.000
10	Decider parent	0.480 [0.221, 0.591]	0.971	0.004
11	Prompt Guard 2	0.464 [0.204, 0.529]	0.989	0.001

Table 4. Test classification on R-Judge.

Rank	Configuration	Macro-F1 [95% CI]	FNR	FPR
1	Jev	0.825 [0.774, 0.873]	0.056	0.297
2	GPT-4.1	0.807 [0.755, 0.857]	0.064	0.324
3	Decider parent	0.604 [0.540, 0.666]	0.232	0.550
4	Laya	0.474 [0.411, 0.536]	0.752	0.207
5	Nimble parent	0.450 [0.389, 0.516]	0.128	0.847
6	Qwen3-8B	0.427 [0.369, 0.487]	0.064	0.901
7	Nimble	0.373 [0.323, 0.424]	0.160	0.928
8	Decider	0.368 [0.313, 0.422]	0.904	0.171

Table 5. Test classification on AgentHarm.

Rank	Configuration	Macro-F1 [95% CI]	FNR	FPR
1	Jev	0.840 [0.774, 0.903]	0.080	0.239
2	Nimble	0.831 [0.756, 0.892]	0.068	0.267
3	Decider	0.778 [0.702, 0.846]	0.415	0.011
4	Nimble parent	0.763 [0.694, 0.824]	0.057	0.403
5	Qwen3-8B	0.751 [0.683, 0.813]	0.034	0.443
6	GPT-4.1	0.741 [0.659, 0.810]	0.006	0.483
7	Laya	0.731 [0.672, 0.792]	0.352	0.182
8	WildGuard	0.584 [0.492, 0.666]	0.023	0.710
9	Decider parent	0.427 [0.370, 0.493]	0.903	0.017

zero. DeepSeek’s difference is 0.021 with CI $[-0.008, 0.057]$. Results vary substantially among the twelve source families, which produces wide intervals despite the larger number of individual inputs.

On AgentHarm, Kimi misses none of 176 harmful requests but flags 96 benign requests as unsafe, compared with Jev’s 14 misses and 42 false alarms. DeepSeek and Qwen3.8 also miss fewer harmful

Table 6. WAIInjectBench classification with reasoning judges.

Rank	Configuration	Macro-F1 [95% CI]	FNR	FPR
1	GLM-5.3-Flash	0.789 [0.495, 0.949]	0.513	0.011
2	Nimble	0.778 [0.495, 0.936]	0.473	0.035
3	DeepSeek-V4.1-Flash	0.777 [0.468, 0.953]	0.548	0.005
4	Jev	0.756 [0.452, 0.937]	0.599	0.001
5	Qwen3.8-27B	0.735 [0.448, 0.923]	0.627	0.005
6	GLM-5.3	0.717 [0.447, 0.946]	0.659	0.005
7	GPT-4.1	0.707 [0.428, 0.932]	0.681	0.002
8	Kimi-K2.6	0.697 [0.425, 0.915]	0.695	0.002

Table 7. R-Judge classification with reasoning judges.

Rank	Configuration	Macro-F1 [95% CI]	FNR	FPR
1	DeepSeek-V4.1-Flash	0.966 [0.941, 0.987]	0.032	0.036
2	Qwen3.8-27B	0.945 [0.915, 0.970]	0.080	0.027
3	GLM-5.3	0.919 [0.885, 0.953]	0.128	0.027
4	GLM-5.3-Flash	0.911 [0.872, 0.948]	0.096	0.081
5	Kimi-K2.6	0.906 [0.865, 0.941]	0.064	0.126
6	Jev	0.825 [0.774, 0.873]	0.056	0.297
7	GPT-4.1	0.807 [0.755, 0.857]	0.064	0.324
8	Nimble	0.373 [0.323, 0.424]	0.160	0.928

Table 8. AgentHarm classification with reasoning judges.

Rank	Configuration	Macro-F1 [95% CI]	FNR	FPR
1	Jev	0.840 [0.774, 0.903]	0.080	0.239
2	Nimble	0.831 [0.756, 0.892]	0.068	0.267
3	GLM-5.3-Flash	0.817 [0.751, 0.874]	0.091	0.273
4	GLM-5.3	0.797 [0.732, 0.853]	0.057	0.341
5	DeepSeek-V4.1-Flash	0.779 [0.705, 0.847]	0.023	0.403
6	Qwen3.8-27B	0.748 [0.670, 0.814]	0.011	0.466
7	GPT-4.1	0.741 [0.659, 0.810]	0.006	0.483
8	Kimi-K2.6	0.705 [0.622, 0.777]	0.000	0.545

requests but falsely flag more benign ones than Jev. This trade-off concerns predicted labels on AgentHarm.



Finding 3. Some judges miss fewer harmful requests but flag more benign ones.

4.4 Model-Specific and Shared Input Populations

Every evaluated configuration returns a valid label for each applicable test input under the evaluated input and scoring settings. The model-specific and shared populations therefore coincide: 1,612 WAIInjectBench inputs, 236 R-Judge records, and 352 AgentHarm requests (Tables 9 and 10). On R-Judge, Jev’s Macro-F1 is 0.825 and GPT-4.1’s is 0.807. Comparisons use the same complete input population within each task.

Table 9. WAIInjectBench results on all supported and shared inputs.

Configuration	<i>N</i>	Macro-F1		FNR		FPR	
	All/shared	All	Shared	All	Shared	All	Shared
Nimble	1612/1612	0.778	0.778	0.473	0.473	0.035	0.035
Qwen3-8B	1612/1612	0.764	0.764	0.495	0.495	0.039	0.039
Jev	1612/1612	0.756	0.756	0.599	0.599	0.001	0.001
GPT-4.1	1612/1612	0.707	0.707	0.681	0.681	0.002	0.002
ProtectAI	1612/1612	0.658	0.658	0.724	0.724	0.023	0.023
PIGuard	1612/1612	0.656	0.656	0.735	0.735	0.018	0.018
Laya	1612/1612	0.649	0.649	0.659	0.659	0.075	0.075
Nimble parent	1612/1612	0.628	0.628	0.796	0.796	0.003	0.003
Decider	1612/1612	0.503	0.503	0.950	0.950	0.000	0.000
Decider parent	1612/1612	0.480	0.480	0.971	0.971	0.004	0.004
Prompt Guard 2	1612/1612	0.464	0.464	0.989	0.989	0.001	0.001

Table 10. R-Judge results on all supported and shared inputs.

Configuration	<i>N</i>	Macro-F1		FNR		FPR	
	All/shared	All	Shared	All	Shared	All	Shared
Jev	236/236	0.825	0.825	0.056	0.056	0.297	0.297
GPT-4.1	236/236	0.807	0.807	0.064	0.064	0.324	0.324
Decider parent	236/236	0.604	0.604	0.232	0.232	0.550	0.550
Laya	236/236	0.474	0.474	0.752	0.752	0.207	0.207
Nimble parent	236/236	0.450	0.450	0.128	0.128	0.847	0.847
Qwen3-8B	236/236	0.427	0.427	0.064	0.064	0.901	0.901
Nimble	236/236	0.373	0.373	0.160	0.160	0.928	0.928
Decider	236/236	0.368	0.368	0.904	0.904	0.171	0.171

4.5 Comparing Misses and False Alarms

We compare each System One model with a control on the same inputs to determine whether it misses fewer unsafe inputs or falsely flags fewer benign ones. Figure 4 shows the 40 paired comparisons. Each point is the System One model’s error rate minus the control’s rate in percentage points, so values below zero favor the System One model. For example, Jev’s -24.4 -point FPR difference from GPT-4.1 on AgentHarm means that it falsely flags 24.4 percentage points fewer benign requests.

Lines show paired 95% cluster-bootstrap confidence intervals, and filled markers identify differences supported by the Holm-adjusted tests described in Section 3. The counts beside model pairs give their shared input totals. FNR uses the unsafe inputs within each pair and FPR uses the benign inputs.

On WAIInjectBench, none of the FNR or FPR differences is significant after adjustment at 0.05, and the small number of source clusters leaves substantial uncertainty. On AgentHarm, the evidence is clearer that Jev flags fewer benign requests than GPT-4.1, while their difference in harmful misses remains uncertain after correction.

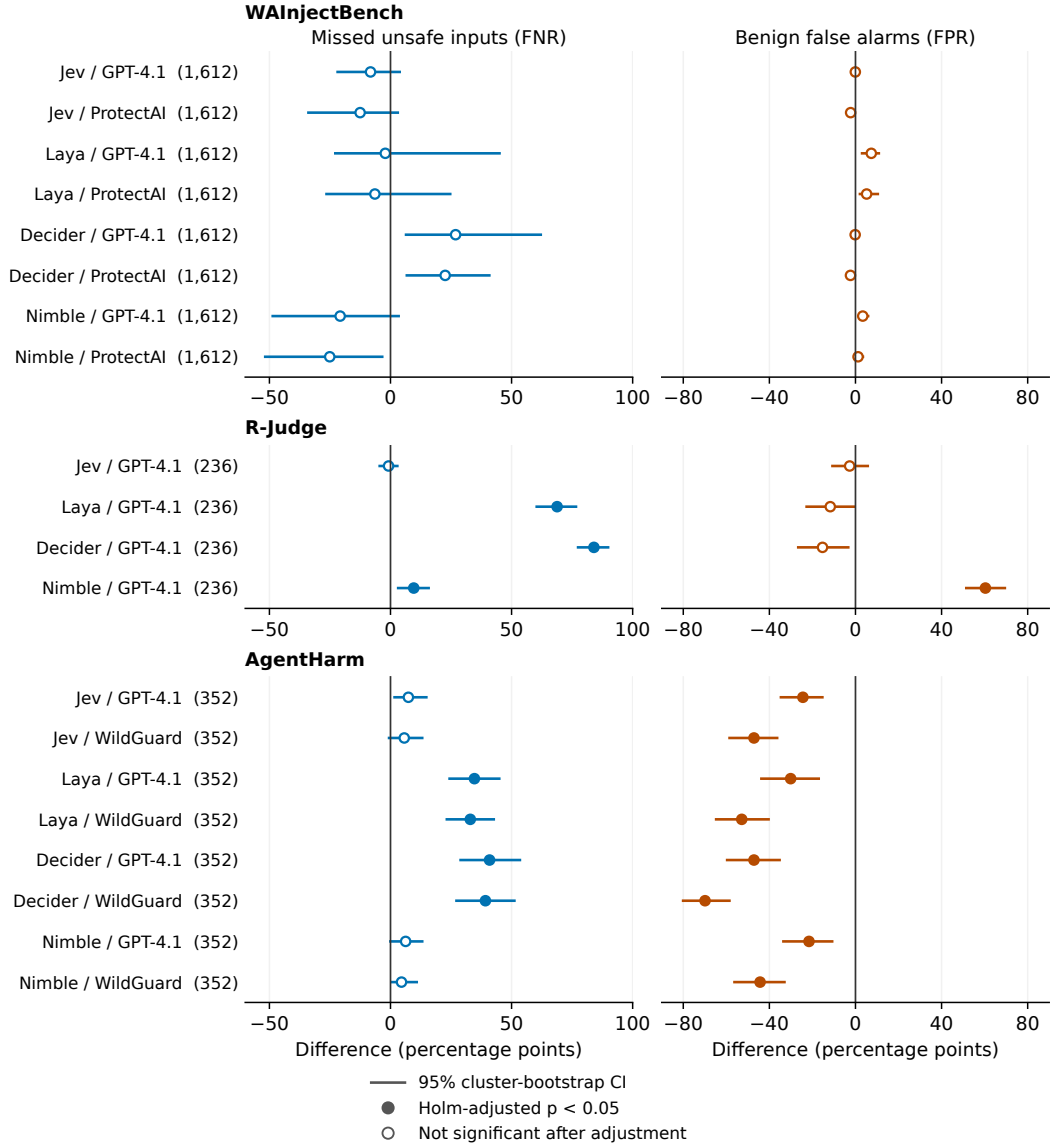


Fig. 4. Differences in missed unsafe inputs and benign false alarms.

5 RQ2: Probability Calibration

Among inputs assigned an unsafe probability near 0.8, about 80% should be unsafe if the predictions are calibrated. RQ2 examines this agreement across tasks and attack groups, then tests whether recalibration fitted on development data improves test probabilities.

5.1 Probability Scores and Calibration Measures

For System One models, we use the unsafe probabilities returned by their decision interfaces. For general judges we normalize the two label-token probabilities at the final answer position after

any generated reasoning and use the unsafe token’s share as the score. GPT-4.1 and Qwen3-8B use separate response configurations for classification and probability analyses, so their predicted labels can differ between these analyses. Recalibration and policy evaluation use the same scoring rule across all partitions. We retain the first usable label and score from the documented response sequence without consulting ground-truth labels (Section 7.1).

For input i , let p_i be the unsafe probability under evaluation and y_i its original label, with 1 denoting unsafe and 0 benign. Brier score is the mean of $(p_i - y_i)^2$, and negative log likelihood (NLL) is the mean binary log loss, computed after clipping probabilities to $[10^{-7}, 1 - 10^{-7}]$. Expected calibration error (ECE) compares mean probability with observed unsafe frequency in ten equal-width probability bins. Reliability diagrams plot these two values, so points above the diagonal indicate underestimated unsafe frequencies. Lower Brier, NLL, and ECE values are better.

We compare the probabilities returned by each configuration, called original scores, with scores recalibrated by temperature scaling [13]. After clipping, we divide the score’s log odds by a temperature T and convert back to a probability. We fit $T \in [0.05, 20]$ by minimizing NLL on development inputs from both classes. Original and recalibrated scores are evaluated on the same inputs, with model weights unchanged.

5.2 Calibration and High-Confidence Errors

Calibration summaries can hide errors in particular input groups and poor separation of safe and unsafe inputs. Table 11 uses the same RQ1 shared test inputs for every row within a task, with N_p denoting the number of usable probabilities. The dagger identifies the GPT-4.1 and Qwen3-8B configurations used for probability evaluation. Figure 5 compares predicted and observed unsafe frequencies across probability bins, with larger markers indicating more inputs.

Among 1,446 WAIInjectBench inputs assigned unsafe probabilities below 0.1, 133 (9.20%) are labeled unsafe, versus a mean predicted probability of 0.205%. Of these 133 unsafe inputs, 130 belong to groups without explicit attack instructions (no-EI). Excluding those groups reduces the unsafe fraction to 0.228% and the mean prediction to 0.176%. This exploratory comparison locates the mismatch with benchmark labels; it does not establish the model’s reasoning or the cause of its errors.



Finding 4. Attacks without direct instructions dominate Jev’s low-score false negatives.

Calibration should be checked alongside separation of safe and unsafe inputs. On shared R-Judge inputs, Laya’s AUROC is 0.533 versus Jev’s 0.961, and its Brier score is 0.272 versus 0.103. Laya’s Brier loss is 9.3% higher than assigning every input the same probability, equal to the test set’s unsafe proportion. This is a descriptive reference computed from test labels. Laya’s scores provide little discrimination between the classes and have higher squared error than this reference.

Binning and constant-probability baselines. Motivated by Guo et al.’s calibration analysis [14], we check ECE with five, ten, and twenty equal-width bins (W5, W10, W20). We also use quantile bins (Q10) that aim for ten equally populated groups but may yield fewer bins because identical scores stay together. Let $\bar{p}$ be the mean unsafe probability and $\bar{y}$ the observed unsafe proportion. Table 12 reports signed bias $\bar{p} - \bar{y}$ and Brier skill $1 - \text{Brier}/[\bar{y}(1 - \bar{y})]$. Positive Brier skill means lower squared error than assigning $\bar{y}$ to every input.

Changing the bins affects the models differently. Jev’s WAIInjectBench ECE changes little, and Laya’s R-Judge ECE is unchanged across the three equal-width choices. Laya’s near-chance R-Judge AUROC measures a separate limitation: weak ranking of unsafe above benign inputs. Brier skill is negative for Laya and Decider on WAIInjectBench and for Laya, Decider, and Nimble on R-Judge.

Table 11. Discrimination and calibration on test inputs.

Configuration	N_p	AUROC $\uparrow$	AP $\uparrow$	Brier $\downarrow$	ECE $\downarrow$	NLL $\downarrow$
WAIInjectBench						
Nimble	1612	0.734	0.582	0.103	0.087	0.364
Jev	1612	0.796	0.662	0.097	0.105	1.140
ProtectAI	1612	0.797	0.565	0.139	0.138	1.234
PIGuard	1612	0.657	0.483	0.132	0.126	0.933
Laya	1612	0.633	0.345	0.147	0.108	0.469
Nimble parent	1612	0.731	0.642	0.116	0.114	0.500
Decider	1612	0.710	0.584	0.148	0.143	0.582
Decider parent	1612	0.616	0.268	0.179	0.201	0.548
Prompt Guard 2	1612	0.784	0.427	0.171	0.171	1.181
Qwen3-8B †	1612	0.714	0.608	0.115	0.117	1.425
GPT-4.1 †	1612	0.811	0.679	0.122	0.124	1.743
GLM-5.3-Flash	1612	0.817	0.660	0.096	0.095	0.966
R-Judge						
Jev	236	0.961	0.975	0.103	0.137	0.318
Decider parent	236	0.570	0.534	0.258	0.156	0.714
Laya	236	0.533	0.532	0.272	0.167	0.741
Nimble parent	236	0.674	0.765	0.373	0.362	1.321
Nimble	236	0.437	0.505	0.320	0.252	0.875
Decider	236	0.610	0.572	0.423	0.440	1.262
Qwen3-8B †	236	0.489	0.577	0.449	0.450	6.338
GPT-4.1 †	236	0.945	0.959	0.138	0.145	1.163
GLM-5.3-Flash	236	0.951	0.963	0.055	0.044	0.312
AgentHarm						
Jev	352	0.938	0.940	0.122	0.128	0.481
Nimble	352	0.935	0.958	0.123	0.170	0.390
Decider	352	0.918	0.937	0.153	0.173	0.497
Nimble parent	352	0.943	0.961	0.172	0.189	0.568
Laya	352	0.827	0.830	0.176	0.072	0.530
WildGuard	352	0.931	0.945	0.325	0.343	1.565
Decider parent	352	0.732	0.743	0.243	0.128	0.679
Qwen3-8B †	352	0.927	0.934	0.237	0.242	2.647
GPT-4.1 †	352	0.931	0.908	0.238	0.241	3.244
GLM-5.3-Flash	352	0.818	0.725	0.160	0.153	1.159

Their probabilities have higher squared error than assigning every input the test set’s unsafe proportion. Matching the average unsafe rate therefore does not ensure useful probabilities for individual inputs.

For confidence comparisons, we use the larger class probability $\max(p_i, 1 - p_i)$. At values ≥ 0.95 , WAIInjectBench false-negative counts are 130 for Jev, 162 for Decider, one for Nimble, and zero for Laya. Jev instead makes four false negatives and ten false positives at this confidence on AgentHarm. Laya reaches this confidence on only three WAIInjectBench inputs, all correct. These counts differ by model and task.

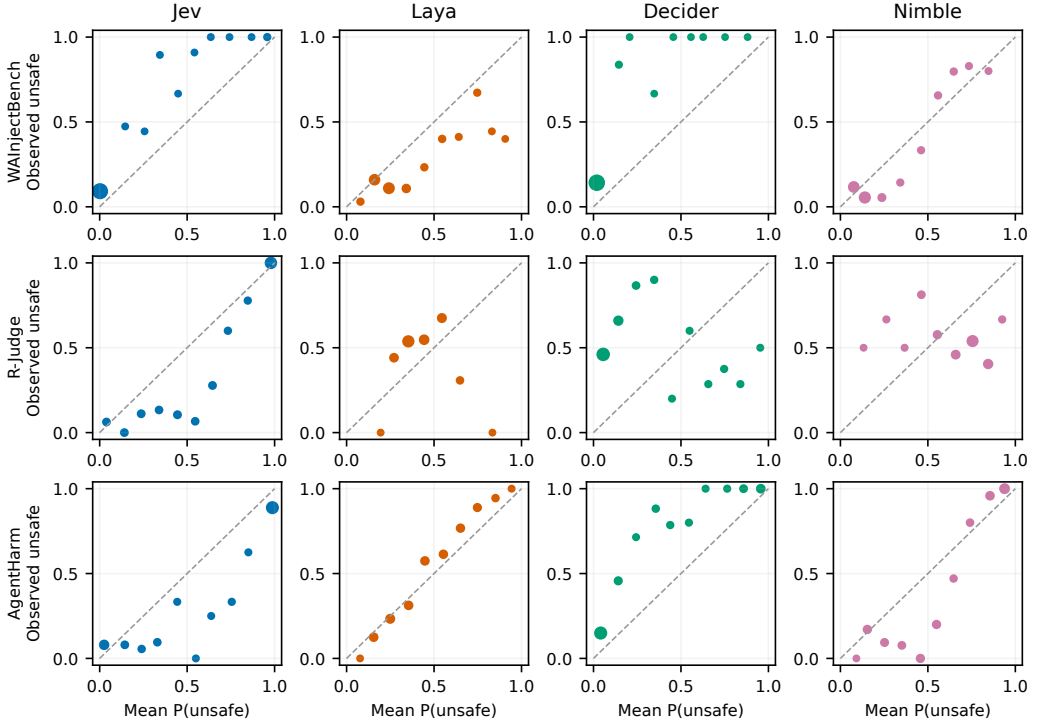


Fig. 5. Calibration of unsafe probabilities on test inputs.

Table 12. Calibration binning and constant-probability baselines.

Component	Bias	W5	W10	W20	Q10	Skill
WAIInjectBench ($N = 1,612$, $\bar{y} = 0.173$)						
Jev	-0.105	0.105	0.105	0.105	0.105	+0.324
Laya	+0.108	0.108	0.108	0.116	0.115	-0.024
Decider	-0.143	0.143	0.143	0.143	0.143	-0.031
Nimble	+0.036	0.052	0.087	0.088	0.090	+0.282
R-Judge ($N = 236$, $\bar{y} = 0.530$)						
Jev	+0.116	0.129	0.137	0.145	0.132	+0.588
Laya	-0.119	0.167	0.167	0.167	0.162	-0.093
Decider	-0.334	0.438	0.440	0.442	0.428	-0.697
Nimble	+0.168	0.252	0.252	0.265	0.253	-0.286
AgentHarm ($N = 352$, $\bar{y} = 0.500$)						
Jev	+0.105	0.119	0.128	0.130	0.131	+0.510
Laya	-0.045	0.072	0.072	0.085	0.089	+0.295
Decider	-0.173	0.173	0.173	0.173	0.173	+0.387
Nimble	+0.101	0.162	0.170	0.172	0.167	+0.507

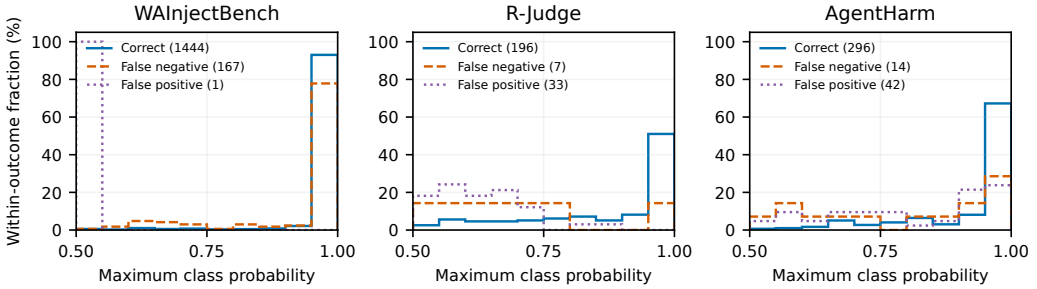


Fig. 6. Maximum class probability by decision outcome for Jev.

Table 13. Discrimination and calibration of reasoning judges.

Configuration	AUROC↑	AP↑	Brier↓	ECE↓	NLL↓
WAIjectBench					
DeepSeek-V4.1-Flash	0.774	0.626	0.099	0.099	1.598
GLM-5.3	0.685	0.531	0.116	0.117	1.187
Kimi-K2.6	0.786	0.642	0.122	0.122	1.580
Qwen3.8-27B	0.645	0.512	0.112	0.112	1.485
R-Judge					
DeepSeek-V4.1-Flash	0.968	0.959	0.034	0.034	0.546
GLM-5.3	0.958	0.974	0.067	0.073	0.444
Kimi-K2.6	0.937	0.920	0.093	0.093	0.856
Qwen3.8-27B	0.954	0.971	0.055	0.055	0.759
AgentHarm					
DeepSeek-V4.1-Flash	0.877	0.844	0.213	0.213	3.433
GLM-5.3	0.813	0.710	0.156	0.131	1.015
Kimi-K2.6	0.922	0.909	0.273	0.273	2.473
Qwen3.8-27B	0.713	0.619	0.239	0.239	3.031

Figure 6 shows maximum class probability separately for correct decisions, false negatives, and false positives, with each histogram normalized within its outcome group.

Reasoning-judge probabilities. The reasoning judges also show that ranking unsafe inputs above safe ones does not ensure accurate probabilities. Table 11 includes GLM-5.3-Flash, while Table 13 reports the remaining reasoning judges on the same shared inputs. Kimi’s AgentHarm AUROC of 0.922 and AP of 0.909 indicate strong ranking, but its Brier and ECE are both 0.273, and it falsely flags 96 benign requests. DeepSeek’s R-Judge Brier loss is 0.034; these judges’ Brier rankings vary across tasks.

5.3 Comparing Adapted Models with Their Base Models

We compare classification and probability metrics for Decider and Nimble against their respective base configurations. Figure 7 shows all twelve paired comparisons on the inputs supported by each pair, with 95% cluster-bootstrap confidence intervals. Both panels put improvements on the right: Macro-F1 gain is adapted minus base, while Brier reduction is base minus adapted.

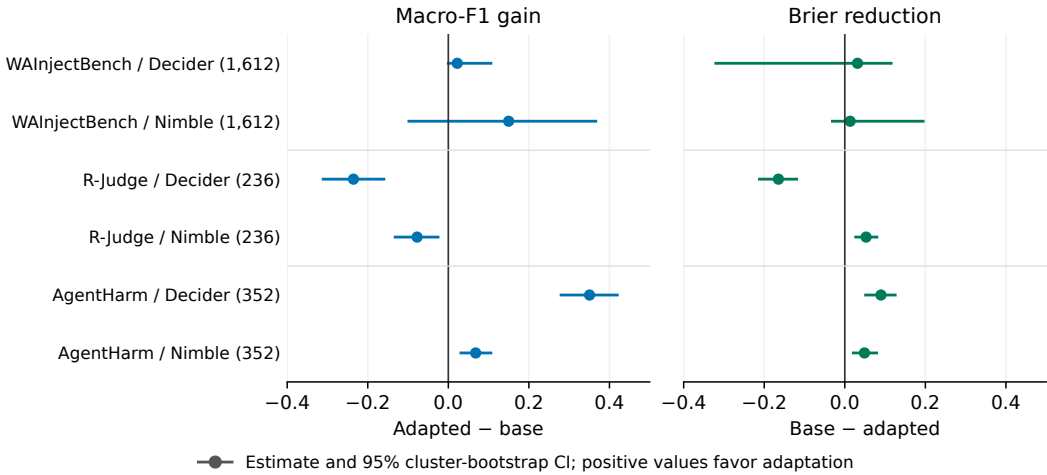


Fig. 7. Classification and probability differences from base configurations.

Both adapted configurations have higher Macro-F1 and lower Brier loss than their base controls on AgentHarm. On R-Judge, Nimble has lower Macro-F1 by 0.078 and lower Brier loss by 0.053, while Decider is worse on both metrics. These comparisons concern the evaluated weights and scoring temperatures, not adaptation in isolation.

5.4 Effects of Recalibration

For each model, we compare original and recalibrated scores on the complete test populations used in the cross-model tables. Temperature scaling improves test NLL in six of the twelve System One model and task combinations, including all four R-Judge comparisons, but the gains do not extend to every metric. On Jev’s 1,612 valid WAIInjectBench outputs, NLL decreases from 1.140 to 0.374 while Brier increases from 0.097 to 0.107.

On AgentHarm, Jev and Nimble classify all 64 development inputs correctly, leading the fit to select a temperature near the lower bound of 0.05 for both. This makes their probabilities more extreme and increases the penalty for high-confidence test errors, with Jev’s NLL rising from 0.481 to 2.163. Recalibration can therefore fit development data well while making test probabilities worse.

Scaling reduces test NLL in all six GPT-4.1 and Qwen3-8B task comparisons using the evaluated scoring configurations. On all R-Judge inputs with usable scores, Qwen’s NLL falls from 6.338 to 0.730 because scaling softens its often near-zero or near-one probabilities, lowering the loss from high-confidence errors.

Table 14 lists the original and recalibrated results, in that order, within each metric cell.

Temperature scaling makes probabilities more or less extreme, but inputs with the same original score still receive the same transformed score. A threshold must therefore handle these tied inputs together, even when some decisions are correct and others are wrong. RQ3 examines how this limits automatic decisions.

6 RQ3: Automation within Error Limits

RQ3 examines the fraction of decisions a policy can automate while keeping unsafe allowances and benign blocks within specified limits, called error budgets. We select thresholds once and keep

Table 14. Test calibration before and after temperature scaling.

Configuration	T	Brier	ECE	NLL
WAIInjectBench				
Jev	10.93	0.097 / 0.107	0.105 / 0.133	1.140 / 0.374
Laya	15.19	0.147 / 0.237	0.108 / 0.310	0.469 / 0.666
Decider	20.00	0.148 / 0.216	0.143 / 0.285	0.582 / 0.625
Nimble	1.18	0.103 / 0.107	0.087 / 0.104	0.364 / 0.376
R-Judge				
Jev	0.90	0.103 / 0.102	0.137 / 0.131	0.318 / 0.316
Laya	20.00	0.272 / 0.250	0.167 / 0.034	0.741 / 0.693
Decider	20.00	0.423 / 0.251	0.440 / 0.087	1.262 / 0.695
Nimble	20.00	0.320 / 0.251	0.252 / 0.034	0.875 / 0.694
AgentHarm				
Jev	0.05	0.122 / 0.156	0.128 / 0.158	0.481 / 2.163
Laya	1.09	0.176 / 0.178	0.072 / 0.077	0.530 / 0.535
Decider	1.67	0.153 / 0.145	0.173 / 0.140	0.497 / 0.453
Nimble	0.05	0.123 / 0.160	0.170 / 0.169	0.390 / 1.454

them fixed for confirmation and test, then compare model errors to assess potential reviewers for deferred inputs.

6.1 Decision Policy and Error Measures

The policy compares unsafe probability p_i with an allow threshold τ_L and a block threshold τ_H :

$$\text{Decision} = \begin{cases} \text{allow,} & p_i < \tau_L, \\ \text{block,} & p_i > \tau_H, \\ \text{escalate,} & p_i \in [\tau_L, \tau_H]. \end{cases} \quad (1)$$

Inputs exactly at either threshold, or without usable scores, also escalate for review. Automatic coverage is the fraction allowed or blocked, while allow coverage counts only allowances. Both use all partition inputs as the denominator. The final evaluated configurations provide usable scores for every input. We report them separately because high coverage can result from blocking many inputs while allowing few to proceed. The unsafe-miss rate $\text{Miss}_{\text{auto}}$ is the fraction of unsafe inputs allowed, and the benign-blocking rate FB_{auto} is the fraction of benign inputs blocked. A low miss rate can therefore come from allowing almost nothing. Allowed-set risk instead measures the unsafe fraction among allowed inputs, with zero-denominator ratios undefined.

6.2 Policy Selection and Independent Confirmation

We search 101 threshold pairs symmetric around 0.5, using evenly spaced $\tau_H \in [0.5, 1]$ and $\tau_L = 1 - \tau_H$. For each allowed miss rate $\epsilon \in \{0.01, 0.05, 0.10\}$, we choose the pair with the highest automatic coverage on selection inputs, subject to $\text{Miss}_{\text{auto}} \leq \epsilon$ and $\text{FB}_{\text{auto}} \leq 0.05$. If coverage is tied, we prefer more automatic allowances, then the larger τ_H . The search includes $(\tau_L, \tau_H) = (0, 1)$, which sends every input for review and automates no decisions.

We apply the selected thresholds unchanged to confirmation and test inputs, counting a policy as passing only when both error rates meet their limits. Policies that fail confirmation are also reported to show whether failures persist on test. These checks use WAIInjectBench and R-Judge, which provide the required partitions (Table 2).

Table 15. Symmetric thresholds at the 1% selection miss limit.

Configuration	Coverage	Allow	Miss	False block	Confirmation pass
WAIInjectBench					
Jev	0.00%	0.00%	0.00%	0.00%	Yes
Laya	5.02%	4.71%	1.08%	0.23%	Yes
Decider	0.00%	0.00%	0.00%	0.00%	Yes
Nimble	0.74%	0.74%	0.36%	0.00%	Yes
Qwen3-8B [‡]	0.00%	0.00%	0.00%	0.00%	Yes
GPT-4.1 [‡]	0.00%	0.00%	0.00%	0.00%	Yes
R-Judge					
Jev	7.63%	0.42%	0.00%	0.00%	Yes
Laya	1.69%	1.27%	0.00%	0.90%	No
Decider	1.69%	1.27%	0.80%	0.00%	Yes
Nimble	6.78%	0.42%	0.00%	6.31%	Yes
Qwen3-8B [‡]	0.00%	0.00%	0.00%	0.00%	Yes
GPT-4.1 [‡]	0.00%	0.00%	0.00%	0.00%	Yes

Small samples and strict error limits. A 1% miss limit requires zero misses among R-Judge’s 74 unsafe selection inputs and 49 unsafe confirmation inputs, because a single miss exceeds the limit in either set. WAIInjectBench has 167 and 111, respectively, permitting one miss in each. With so few permitted errors, individual inputs can determine whether a policy passes, and larger samples are needed to estimate rare-error rates precisely.

The benign-blocking limit is similarly sensitive to individual errors. R-Judge confirmation permits at most two blocks among 44 benign inputs under its 5% limit, while test permits five among 111. Cluster-bootstrap intervals describe uncertainty from the sampled interaction records and attack sources.

6.3 Coverage and Errors of Symmetric Policies

Table 15 reports test results at the 1% selection miss limit. A “Yes” under “Confirmation pass” means that both error limits were met on independent confirmation inputs. The double dagger identifies the GPT-4.1 and Qwen3-8B probability-scoring configurations used for threshold selection.

With a 1% miss limit during selection, symmetric policies send every WAIInjectBench input for review with Jev, Decider, GPT-4.1, and Qwen3-8B (Table 15). Laya automates 5.02% but allows three of 279 unsafe inputs, exceeding the miss limit. On R-Judge, Nimble passes confirmation but blocks seven of 111 benign test inputs, exceeding the 5% blocking limit. Meeting the limits before test evaluation does not ensure they hold on test inputs.

Jev’s 7.63% R-Judge coverage consists of 17 blocks and one allowance, so only 0.42% of inputs proceed without review. GPT-4.1 and Qwen3-8B escalate every R-Judge input with either original or recalibrated scores, showing that their lower NLL after recalibration does not produce automatic decisions under these symmetric rules.

Raising the miss limit used for threshold selection can increase coverage but also change benign blocking (Figure 8). Circles mark policies meeting both test limits, and crosses mark failures. Jev’s R-Judge coverage rises from 7.63% to 72.88% at a 5% miss budget, but benign blocking reaches 5.41%, above its unchanged limit. This policy allows 54 inputs, including three unsafe ones. Those misses

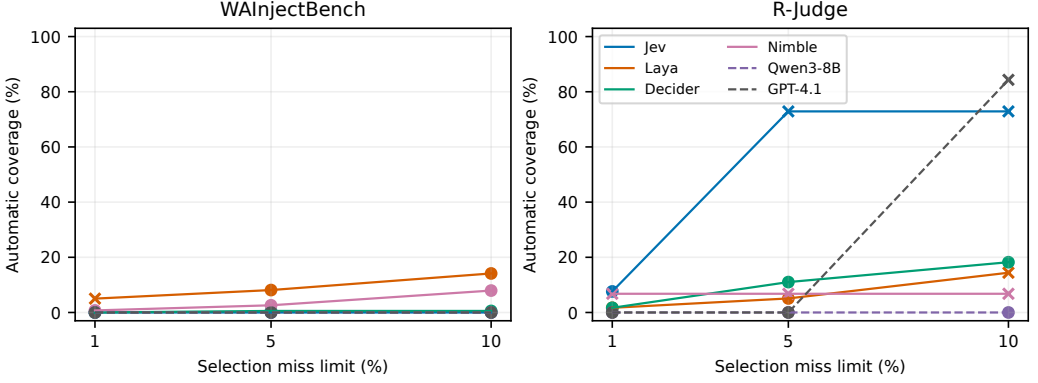


Fig. 8. Automatic coverage at different selection miss limits.

are 2.4% of all unsafe inputs but 5.56% of allowances, illustrating why low overall miss rates need not imply low risk among allowed inputs.

6.4 Independent Allowance and Blocking Thresholds

We examine independent allow and block thresholds by removing the symmetry constraint: choosing a low allow threshold no longer forces a high block threshold. We search all 101×101 combinations of evenly spaced $\tau_L \in [0, 0.5]$ and $\tau_H \in [0.5, 1]$, keeping scores, partitions, error limits, and the coverage objective unchanged. We use the same tie-breaking rules, then prefer the smaller τ_L if a tie remains. We select thresholds on selection inputs and apply them unchanged to confirmation and test, comparing both policy types with original and recalibrated scores at all three miss limits.

Figure 9 compares symmetric (S) and independent (I) thresholds using original scores at the 1% miss limit on WAINjectBench and R-Judge. Stacked bars separate allowances from blocks, and markers show whether policies meet both limits on confirmation and test, fail confirmation only, or fail on test.

On R-Judge, independent thresholds raise Jev’s test coverage from 7.63% to 47.03% while meeting confirmation and test limits, but the policy allows only one input and blocks 110. On WAINjectBench, its coverage rises from zero to 6.70%, entirely through 108 blocks. The number of inputs allowed without review is unchanged in both comparisons.



Finding 5. At the strictest miss limit, separate thresholds add coverage mainly through blocks.

Nimble similarly rises from 0.74% to 12.78% WAINjectBench coverage while allowing the same twelve inputs. Independent thresholds separate the two actions, but higher selection coverage does not ensure that error limits hold on other inputs. GPT-4.1 blocks 128 R-Judge inputs, including 17 benign ones, and fails both confirmation and test checks. Among the twelve model/task pairs with original scores at the 1% miss limit, seven independent policies pass both checks, compared with nine symmetric policies. Several passing symmetric policies send every input for review.

Table 16 shows why higher coverage alone can be misleading. Laya’s independent WAINjectBench policy retains the same 76 allowances and three misses as its symmetric policy, while the lower blocking threshold adds 128 blocks, including 62 additional benign inputs. Coverage therefore rises without correcting the unsafe allowances that already exceeded the miss budget, and more benign inputs are rejected.

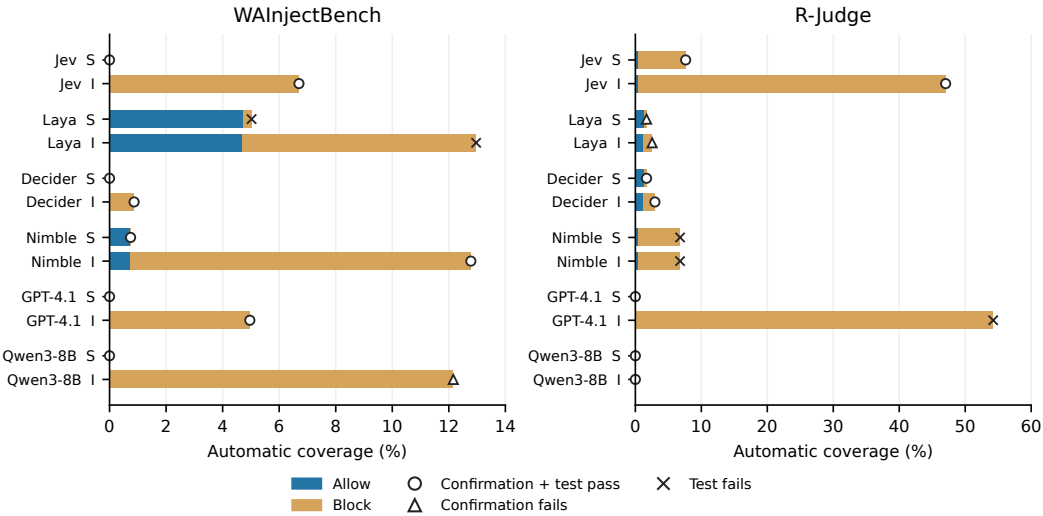


Fig. 9. Automation with symmetric and independent thresholds.

Table 16. Independent thresholds at the 1% selection miss limit.

Configuration	τ_L	τ_H	Allowed	Blocked	Misses	False blocks	C/T
<i>WAINjectBench</i>							
Jev	0.000	0.525	0	108	0	0	Y/Y
Laya	0.105	0.570	76	133	3	65	Y/N
Decider	0.000	0.515	0	14	0	0	Y/Y
Nimble	0.050	0.500	12	194	1	47	Y/Y
GPT-4.1	0.000	0.655	0	80	0	2	Y/Y
Qwen3-8B	0.000	0.560	0	196	0	54	N/Y
<i>R-Judge</i>							
Jev	0.005	0.745	1	110	0	2	Y/Y
Laya	0.220	0.685	3	3	0	3	N/Y
Decider	0.015	0.925	3	4	1	2	Y/Y
Nimble	0.150	0.885	1	15	0	7	Y/N
GPT-4.1	0.000	0.985	0	128	0	17	N/N
Qwen3-8B	0.000	1.000	0	0	0	0	Y/Y

Allowed, blocked, misses, and false blocks are input counts. C/T: confirmation/test budget checks.

At the 1% selection miss limit, recalibrated scores also gain coverage without consistently meeting both error limits (Table 17). On R-Judge, Jev again reaches 47.03% coverage with one allowance, and Decider blocks four inputs that its symmetric policy sent for review. Qwen3-8B gains four allowances but fails confirmation. Across miss limits of 1%, 5%, and 10%, independent policies pass both checks in 7, 9, and 7 of the twelve recalibrated model/task pairs, compared with 10, 9, and 9 symmetric policies.

For Jev’s original scores on R-Judge, independent thresholds increase test coverage by 39.41 percentage points (95% cluster-bootstrap CI [33.05, 45.34]), entirely from additional blocks. The interval describes variation across test samples with the selected thresholds fixed.

Table 17. Independent thresholds after probability recalibration.

Configuration	τ_L	τ_H	Cov. S	Cov. I	Allow I	Miss I	FB I	C/T
<i>WAINjectBench</i>								
Jev	0.000	0.500	0.00	6.95	0.00	0.00	0.08	Y/Y
Laya	0.465	0.505	5.15	12.90	4.84	1.08	4.73	Y/N
Decider	0.000	0.500	0.00	0.87	0.00	0.00	0.00	Y/Y
Nimble	0.075	0.500	0.37	12.41	0.37	0.00	3.53	Y/Y
GPT-4.1	0.000	0.505	0.00	4.96	0.00	0.00	0.15	Y/Y
Qwen3-8B	0.000	0.505	0.00	12.16	0.00	0.00	4.05	N/Y
<i>R-Judge</i>								
Jev	0.005	0.770	7.63	47.03	0.42	0.00	1.80	Y/Y
Laya	0.485	0.510	1.69	2.54	1.27	0.00	2.70	N/Y
Decider	0.000	0.530	0.00	1.69	0.00	0.00	1.80	Y/Y
Nimble	0.480	0.530	2.12	2.97	0.85	0.80	1.80	Y/Y
GPT-4.1	0.000	0.625	0.00	54.24	0.00	0.00	15.32	N/N
Qwen3-8B	0.330	1.000	0.00	1.69	1.69	0.00	0.00	N/Y

Rates are percentages. S/I: symmetric/independent. FB: benign-blocking rate. C/T: confirmation/test budget checks.

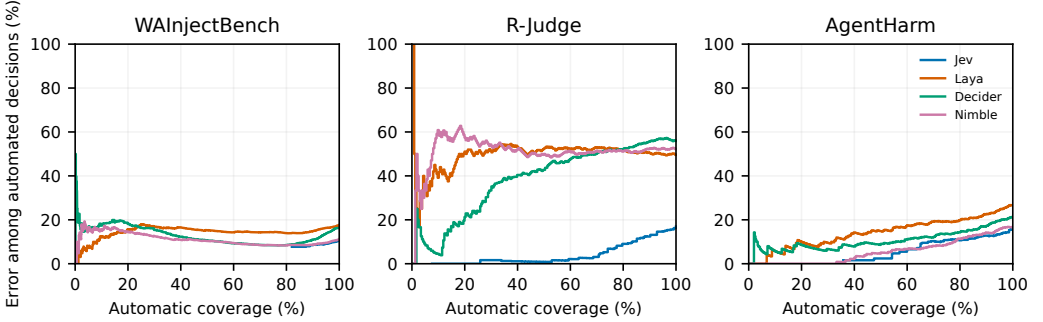


Fig. 10. Error rate versus automatic coverage.

6.5 Confidence Ranking and Identical Scores

To examine how confidence separates correct from incorrect decisions, Figure 10 ranks shared inputs by maximum class probability and accepts predictions from highest to lowest confidence. Both safe and unsafe predictions count as automatic decisions, with equal-confidence inputs added together. Selective error is the fraction of accepted predictions that are wrong.

Thresholds cannot separate correct and incorrect decisions with the same score. Jev assigns $p_i = 0$ to 1,320 shared WAINjectBench inputs, including 102 misses, and $p_i = 1$ to three correct predictions. Accepting these probability-zero and probability-one predictions immediately covers 82.07% at 7.71% error. Any positive allow threshold admits all zero-score inputs, including the missed attacks. An independent block threshold can reject high-score inputs but cannot distinguish attacks from benign inputs within that zero-score group.

Jev’s maximum-confidence AgentHarm group instead covers 126 of 352 inputs with two errors (1.59%). Identical reported confidence therefore corresponds to different error rates across tasks.

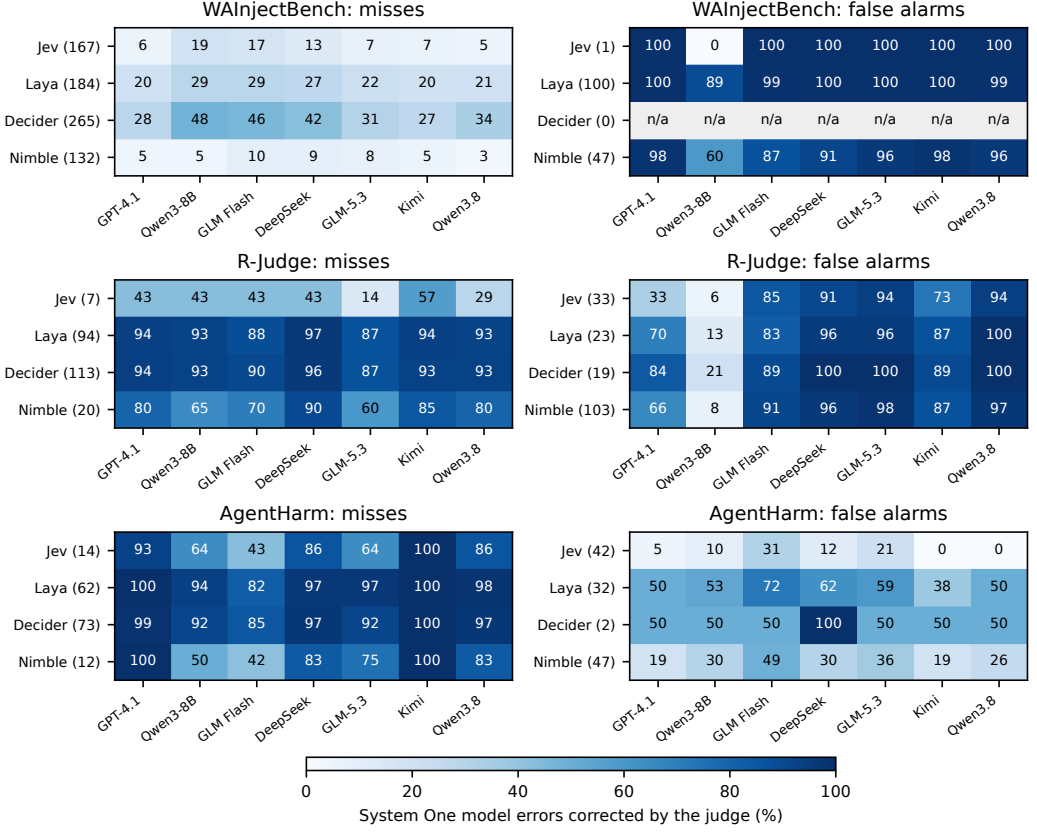


Fig. 11. System One model errors corrected by other judges.

6.6 Other Judges as Potential Reviewers

An application can send uncertain inputs to another judge for review. Motivated by earlier evidence of shared judge errors [26], we compare labels collected separately for the same inputs. A correction means that a judge is correct where the System One model is wrong. This measures error overlap, not the performance of an executed review cascade.

We compare four System One models with seven general judges on 1,612 shared WAIjectBench inputs, 236 R-Judge records, and 352 AgentHarm requests. Adding task-compatible specialists gives 100 model pairs, evaluated on both shared and pair-specific inputs.

For each class, a judge can correct or repeat the System One model’s errors, or introduce errors on inputs that model classified correctly. Figure 11 reports the fraction of the System One model’s errors corrected by each judge. Row counts give the original errors, and grey cells indicate no errors to correct. Table 18 also counts new errors on inputs Jev classified correctly. For example, GPT-4.1 detects 10 of the 167 WAIjectBench attacks missed by Jev but adds 33 misses of its own, leaving 190 misses in total. Thus, correcting some Jev errors need not reduce the judge’s total errors.

On WAIjectBench, Jev misses 167 unsafe inputs, and the seven judges correct between 8 and 32. At $\max(p_i, 1 - p_i) \geq 0.95$, each judge detects at most five of Jev’s 130 misses and 30 of Decider’s 162. Every judge therefore repeats over 80% of these errors for each model. Nimble has one such

Table 18. Errors corrected and added by judges compared with Jev.

Judge	False negatives		False positives	
	Corrected	Added	Corrected	Added
<i>WAIInjectBench</i> , $N = 1,612$				
GPT-4.1	10 of 167	33	1 of 1	2
Qwen3-8B	32 of 167	3	0 of 1	51
GLM-5.3-Flash	29 of 167	5	1 of 1	14
DeepSeek-V4.1-Flash	22 of 167	8	1 of 1	7
GLM-5.3	12 of 167	29	1 of 1	6
Kimi-K2.6	11 of 167	38	1 of 1	3
Qwen3.8-27B	8 of 167	16	1 of 1	6
<i>R-Judge</i> , $N = 236$				
GPT-4.1	3 of 7	4	11 of 33	14
Qwen3-8B	3 of 7	4	2 of 33	69
GLM-5.3-Flash	3 of 7	8	28 of 33	4
DeepSeek-V4.1-Flash	3 of 7	0	30 of 33	1
GLM-5.3	1 of 7	10	31 of 33	1
Kimi-K2.6	4 of 7	5	24 of 33	5
Qwen3.8-27B	2 of 7	5	31 of 33	1
<i>AgentHarm</i> , $N = 352$				
GPT-4.1	13 of 14	0	2 of 42	45
Qwen3-8B	9 of 14	1	4 of 42	40
GLM-5.3-Flash	6 of 14	8	13 of 42	19
DeepSeek-V4.1-Flash	12 of 14	2	5 of 42	34
GLM-5.3	9 of 14	5	9 of 42	27
Kimi-K2.6	14 of 14	0	0 of 42	54
Qwen3.8-27B	12 of 14	0	0 of 42	40

miss, repeated by all seven judges; Laya has none. A rule accepting predictions at that confidence would leave these cases unreviewed.

On AgentHarm, some judges detect previously missed harmful requests but falsely flag more benign ones. Kimi corrects all 14 Jev misses without introducing another miss, but repeats all 42 Jev false positives and adds 54. GPT-4.1 corrects 13 misses and introduces none, while correcting two false positives and adding 45. On shared R-Judge inputs, DeepSeek corrects 30 of Jev’s 33 false positives and three of its seven misses, introducing one false positive and no new miss.

Error correction rates vary across model and task pairs. Every judge corrects over 80% of Laya’s and Decider’s AgentHarm misses, while none corrects 10% of Nimble’s WAIInjectBench misses. These comparisons do not support choosing a review judge from aggregate accuracy alone.

These comparisons identify judges that could correct particular errors, but a policy must send the relevant inputs to them. Reviewing only uncertain inputs leaves high-confidence errors unreviewed, even when another judge could detect them.

7 Discussion

7.1 Component Integration and Error Handling

Input and Service Limitations. All final classification configurations cover the complete test inputs for their assigned tasks (Section 4.2). Input coverage does not establish decision reliability. Network failures, input limits, and invalid answers still require explicit handling when these models are deployed.

Output Format and Score Availability. The scoring protocol retains the first response with a usable label and score, without consulting the reference label. Response formats and hosted or local backends determine how candidate-label scores are obtained. The artifact specifies these choices for each evaluated configuration.

Deployment Measurements. Response time and API charges depend on routing, queueing, caching, and reasoning settings, so comparisons need controlled services, hardware, and workloads. Local backends and numerical precision can also affect execution and scores, and our study does not isolate their effects. AgentDojo and AgentDyn can further measure how blocking or escalating inputs affects attack success and benign-task completion [9, 18].

7.2 Implications for Model Development

Specify what counts as unsafe. Jev’s low-score WAINjectBench errors concentrate in groups without direct attack instructions. Separate questions about instruction conflicts, source trust, and requested harm could test which evidence drives these decisions.

Improve probabilities near the allow and block thresholds. Training could put more weight on unsafe inputs scored low enough to allow and benign inputs scored high enough to block, since these errors directly affect automation. The mixed temperature-scaling results motivate checking improvements on independent data for each task.

Evaluate complete inputs alongside decision quality. Laya covers every input with the evaluated 2,048-token budget but still poorly separates safe from unsafe R-Judge interactions. Input coverage and decision quality require separate checks.

Return probabilities for every candidate label. Returning only the most likely tokens can omit scores needed to compute the class probability. Providing all candidate scores would simplify integration, while recording the preceding text and model revision would help identify changes caused by the service or output format.

8 Threats to Validity

Internal validity. Comparisons depend on prompts, answer-label order, context limits, reasoning settings, and services. Response sampling and output formats can affect scores. Recalibration uses only development data; threshold selection uses only selection data, with no retuning on confirmation or test. Rare errors and few source clusters limit interval precision.

Construct validity. AgentHarm labels harmful requests as unsafe; WAINjectBench labels attack-derived text as unsafe even without explicit instructions. The EI/no-EI, threshold-sensitivity, and cross-model error analyses are exploratory. Our finite grid restricts allowance thresholds to at most 0.5 and blocking thresholds to at least 0.5; other policies may yield different trade-offs.

External validity. Public benchmarks may overlap training data, and unsafe-input proportions affect precision, AP, and calibration. Transfer to adaptive attacks, other languages, and multimodal inputs remains open. Hosted revisions and service behavior can change. Protection also depends on

the agent, environment, guard placement, and handling of blocked requests, requiring validation in the target system.

9 Conclusion

Our offline evaluation shows that aggregate rankings and calibration metrics can obscure concentrated classification errors, and the two adapted models do not improve Macro-F1 consistently across tasks. At the strictest selection limits, separate allow and block thresholds increase automation mainly through additional blocks, while allowances remain limited. Passing confirmation does not ensure the empirical limits hold on test. Potential reviewers also share errors: for Jev and Decider, each judge detects fewer than one fifth of high-confidence WAIInjectBench misses. These results support evaluating errors within attack groups, probability quality, and fixed-policy allow/block/review outcomes alongside aggregate performance when selecting models for agent security.

Acknowledgments

OpenAI Codex assisted with study design and code writing.

Data Availability

The artifact provides experiment code, final predictions, benchmark partitions, configurations, and reproducible analyses (<https://github.com/yxsec/system-one-security-eval>).

References

- [1] Saleema Amershi, Andrew Begel, Christian Bird, Robert DeLine, Harald Gall, Ece Kamar, Nachi Nagappan, Besmira Nushi, and Tom Zimmermann. 2019. Software Engineering for Machine Learning: A Case Study. In *Proceedings of the International Conference on Software Engineering: Software Engineering in Practice*. IEEE. <https://www.microsoft.com/en-us/research/publication/software-engineering-for-machine-learning-a-case-study/>
- [2] Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, Yarin Gal, and Xander Davies. 2025. AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents. In *International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2025/hash/c493d23af93118975cdbc32cbe7323f5-Abstract-Conference.html
- [3] Anastasios N. Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. 2024. Conformal Risk Control. In *International Conference on Learning Representations*. <https://research.google/pubs/conformal-risk-control/>
- [4] Bespoke Labs and Maheswaran Sathiamoorthy. 2026. Nimble. <https://github.com/bespokelabsai/nimble> Official implementation; evaluated model: Bespoke-Nimble-9B.
- [5] Md Anas Biswas. 2026. Confidently Wrong: Severity-Aware Calibration of Prompt-Injection Detectors under Attack Shift. arXiv:2606.22659 [cs.CR] <https://arxiv.org/abs/2606.22659>
- [6] Eric Breck, Shaoqing Cai, Eric Nielsen, Michael Salib, and D. Sculley. 2017. The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction. In *Proceedings of the IEEE International Conference on Big Data*. IEEE. <https://research.google/pubs/the-ml-test-score-a-rubric-for-ml-production-readiness-and-technical-debt-reduction/>
- [7] convaiinnovations. [n. d.]. *Laya: Model Card and Checkpoint*. <https://huggingface.co/convaiinnovations/laya> Model card accessed September 24, 2026; evaluated revision recorded in the artifact.
- [8] Edoardo DeBenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813 [cs.CR] <https://arxiv.org/abs/2503.18813>
- [9] Edoardo DeBenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems*, Vol. 37. doi:10.52202/079017-2636
- [10] Yonatan Geifman and Ran El-Yaniv. 2017. Selective Classification for Deep Neural Networks. In *Advances in Neural Information Processing Systems*, Vol. 30. https://papers.neurips.cc/paper_files/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cf1-Abstract.html

- [11] Yonatan Geifman and Ran El-Yaniv. 2019. SelectiveNet: A Deep Neural Network with an Integrated Reject Option. In *Proceedings of the 36th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 97)*. PMLR, 2151–2159. <https://proceedings.mlr.press/v97/geifman19a.html>
- [12] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. arXiv:2302.12173 [cs.CR] <https://arxiv.org/abs/2302.12173>
- [13] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On Calibration of Modern Neural Networks. In *Proceedings of the 34th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 70)*. PMLR, 1321–1330. <https://proceedings.mlr.press/v70/guo17a.html>
- [14] Ruoqi Guo, Yi Liu, Gelei Deng, Yuekang Li, Lida Zhao, Yutao Wu, Simin Chen, Ying Zhang, and Leo Yu Zhang. 2026. Just Ask Jev: Reinforcement Learning for Calibrated Decisions as a Zero-Shot Detector of AI Alignment Failures. arXiv:2609.29429 [cs.AI] <https://arxiv.org/abs/2609.29429v1>
- [15] Seungju Han, Kavel Rao, Allyson Ettinger, Liwei Jiang, Bill Yuchen Lin, Nathan Lambert, Yejin Choi, and Nouha Dziri. 2024. WildGuard: Open One-Stop Moderation Tools for Safety Risks, Jailbreaks, and Refusals of LLMs. In *Advances in Neural Information Processing Systems*, Vol. 37. doi:10.52202/079017-0261
- [16] Jiaju Huang, Hao Yang, Xinyu Ma, Xinglong Liang, Kunyan Cai, Junqiang Ma, Shaobin Chen, Yue Sun, and Tao Tan. 2026. Can Jev Judge Radiology Reports? Evaluating a System One Model for Clinical Factuality. arXiv:2609.27607 [cs.CL] <https://arxiv.org/abs/2609.27607v1>
- [17] leolee99. [n. d.]. *PIGuard: Model Card and Checkpoint*. <https://huggingface.co/leolee99/PIGuard> Model card accessed September 24, 2026.
- [18] Hao Li, Ruoyao Wen, Shanghao Shi, Ning Zhang, Yevgeniy Vorobeychik, and Chaowei Xiao. 2026. AgentDyn: Are Your Agent Security Defenses Deployable in Real-World Dynamic Environments? arXiv:2602.03117 [cs.CR] <https://arxiv.org/abs/2602.03117>
- [19] Yubo Li, Yidi Miao, Ramayya Krishnan, and Rema Padman. 2026. JEV-as-a-Judge: Accept When Confident, Escalate When Unsure. arXiv preprint arXiv:2609.26550 (2026). <https://arxiv.org/abs/2609.26550v1> Version 1.
- [20] Guoming Ling, Muen Xue, and Zijian Ye. 2026. Jev in the Wild: A Data-Driven Analysis of the Jev Model’s Functionality, Applications and Ecosystem. arXiv:2609.30216 [cs.SE] <https://arxiv.org/abs/2609.30216v1>
- [21] Yinuo Liu, Xilong Wang, Ruohan Xu, Yuqi Jia, and Neil Zhenqiang Gong. 2026. WAIInjectBench: Benchmarking Prompt Injection Detections for Web Agents. arXiv:2510.01354v2 [cs.CR] <https://arxiv.org/abs/2510.01354v2>
- [22] Mapika. [n. d.]. *Decider-2B: Model Card and Checkpoint*. <https://huggingface.co/Mapika/decider-2b> Model card accessed September 24, 2026; evaluated revision recorded in the artifact.
- [23] Meta. [n. d.]. *Llama Prompt Guard 2 86M: Model Card*. <https://huggingface.co/meta-llama/Llama-Prompt-Guard-2-86M> Model card accessed September 24, 2026.
- [24] Yaniv Ovadia, Emily Fertig, Jie Ren, Zachary Nado, D. Sculley, Sebastian Nowozin, Joshua Dillon, Balaji Lakshminarayanan, and Jasper Snoek. 2019. Can You Trust Your Model’s Uncertainty? Evaluating Predictive Uncertainty under Dataset Shift. In *Advances in Neural Information Processing Systems*, Vol. 32. https://papers.nips.cc/paper_files/paper/2019/hash/8558cb408c1d76621371888657d2eb1d-Abstract.html
- [25] Protect AI. [n. d.]. *DeBERTa-v3-base-prompt-injection-v2: Model Card*. <https://huggingface.co/protectai/deberta-v3-base-prompt-injection-v2> Model card accessed September 24, 2026.
- [26] Delip Rao and Chris Callison-Burch. 2026. JEV vs. LLMs as Rubric Judges: Cheaper, Faster, and Wrong in the Same Places. arXiv preprint arXiv:2609.29769 (2026). <https://arxiv.org/abs/2609.29769> Version 1.
- [27] Yusuf Khalid Shire and Sang-Chul Kim. 2026. PIDS-Bench: Evaluating Prompt-Injection Detectors Under Over-Defense, Obfuscation, and Distribution Shift. *IEEE Access* 14 (2026), 134184–134205. doi:10.1109/ACCESS.2026.3728186
- [28] Yu Sun, Junhao Xu, Jiajia Shi, and Zijin Yang. 2026. Type-Safe Is Not Error-Free: Typed Decision Models Follow the Option Name, Not the Definition Bound to It. arXiv:2609.26758v2 [cs.AI] <https://arxiv.org/html/2609.26758v2>
- [29] TypeSafe. [n. d.]. *Jev Documentation: Confidence*. <https://docs.typesafe.ai/confidence> Documentation accessed September 24, 2026.
- [30] Haoyu Wang, Christopher M. Poskitt, Jiali Wei, and Jun Sun. 2026. ProbGuard: Proactive Runtime Monitoring for LLM Agent Safety via Probabilistic Prediction. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. ACM. <https://arxiv.org/abs/2508.00500v4> To appear.
- [31] willkelly. 2026. Evaluating Jev. <https://github.com/willkelly/jev-evaluation/tree/d80f375621ad4b9306c6dff6941242925d7e2386> Public evaluation repository and technical report; revision d80f375, accessed September 24, 2026.
- [32] Tiantong Wu and Wei Yang Bryan Lim. 2026. Decision Hijacking: Prompt Injection Attacks on Jev’s Typed Probabilistic Decisions. arXiv:2609.28613 [cs.CR] <https://arxiv.org/abs/2609.28613v1>
- [33] Tiantong Wu and Wei Yang Bryan Lim. 2026. REFLEX with Jev for Efficient Selective Control in LLM Agents. arXiv:2609.26532 [cs.AI] <https://arxiv.org/abs/2609.26532v1>

- [34] Zixiang Xu. 2026. JevOut: Natural Context Can Flip Decision Models. arXiv:2609.30243 [cs.CL] <https://arxiv.org/abs/2609.30243v1>
- [35] Tongxin Yuan, Zhiwei He, Lingzhong Dong, Yiming Wang, Ruijie Zhao, Tian Xia, Lizhen Xu, Binglin Zhou, Fangqi Li, Zhuosheng Zhang, Rui Wang, and Gongshen Liu. 2024. R-Judge: Benchmarking Safety Risk Awareness for LLM Agents. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 1467–1490. doi:10.18653/v1/2024.findings-emnlp.79
- [36] Fan Zhang, Yankai Chen, Zhuohan Xie, Yixi Zhou, Sijia Peng, Lei Fan, Xinhua Ji, Cunyuan Zheng, Huangyong Shan, Philip S. Yu, Xue Liu, Yu Chen, Preslav Nakov, and Songwei He. 2026. Same Scores, Different Decisions: Evaluating JEV and Language Models for Legal Document Understanding. arXiv:2609.27678 [cs.CL] <https://arxiv.org/abs/2609.27678>